LangChain Prompts – Complete Notes

What is LangChain Prompt?

A LangChain Prompt is a structured template used to send formatted instructions to a Large Language Model (LLM) inside the LangChain framework.

Instead of sending plain text directly to an LLM, LangChain uses prompt templates to:

- Organize instructions clearly
- Insert dynamic variables (like user questions)
- Control model behavior
- Build reusable and scalable AI applications

In Simple Words

A LangChain prompt is:

A programmable instruction template that tells the LLM how to respond and what data to use.

Why LangChain Prompts Are Important

They help to:

- Standardize AI responses
- Add user input dynamically
- Combine retrieved data (in RAG systems)
- Separate system instructions and user queries
- Build chatbots and AI agents

Types of Prompts in LangChain

1. **PromptTemplate** – Used for text-based models
2. **ChatPromptTemplate** – Used for chat-based models (system, human, assistant roles)

Example Concept

Basic prompt idea:

"You are an academic tutor. Answer the question clearly: {question}"

Here:

- "You are an academic tutor" → System instruction
- **{question}** → Dynamic user input

LangChain replaces the variable automatically before sending it to the model.

In AI Applications

LangChain prompts are widely used in:

- AI Agents
- RAG (Retrieval-Augmented Generation) systems
- Chatbots
- Question-answering systems
- Intelligent tutoring systems

Final Definition

LangChain Prompt is a structured and reusable template used in the LangChain framework to format instructions and dynamically pass data to Large Language Models for generating controlled and context-aware responses.

LangChain Prompts Without Users

1 Definition

LangChain Prompts Without Users refer to prompt templates that operate independently without any user interaction. These prompts are fully system-driven and are executed without involving a human query or conversation.

In this case, the system itself defines the instruction, and the Large Language Model (LLM) generates the output based solely on that predefined instruction.

2 Core Concept

In prompts without users:

- There is no human participation.
- No interactive conversation occurs.
- The prompt is predefined by the developer or system.
- The execution is automated.

The system controls what the model generates.

3 Example

```
from langchain.prompts import PromptTemplate

prompt = PromptTemplate(

    input_variables=[],

    template="Generate a summary of Artificial Intelligence."

)
```

Explanation:

- The instruction is fixed.
- No user is involved.
- The model produces output automatically.

4️⃣ Where It Is Used

LangChain prompts without users are used in:

- Automated report generation
- Background AI processing
- Scheduled content creation
- Internal system instructions
- Batch data summarization

5️⃣ Characteristics

- Static and predefined
- Fully system-controlled
- No human interaction
- Non-interactive
- Reusable for automation

6️⃣ Advantages

- Easy to implement
- Consistent output behavior
- Suitable for automation
- Simple debugging and testing

7️⃣ Limitations

- No personalization
- Not suitable for chat applications
- Cannot handle real-time queries

Final Exam Definition

LangChain Prompts Without Users are predefined and automated prompt templates that generate responses from a Large Language Model without involving any user interaction or conversational input.

LangChain Prompts With Users

1️⃣ Definition

LangChain Prompts with Users refer to prompt templates that involve user interaction. In this type, the prompt dynamically incorporates user queries or messages before being sent to the Large Language Model (LLM).

The system receives input from the user and inserts it into the prompt template at runtime.

2️⃣ Core Concept

In prompts with users:

- A user provides a query or message.
- The prompt contains placeholders (variables).
- The system replaces placeholders with user data.
- The LLM generates a response based on the combined instruction and user input.

This makes the prompt dynamic and interactive.

3️⃣ Example Using PromptTemplate

```
from langchain.prompts import PromptTemplate

prompt = PromptTemplate(

    input_variables=["question"],

    template="Answer the following question clearly: {question}"

)
```

Explanation:

- question is provided by the user.
- {question} is replaced at runtime.
- The model generates a personalized response.

4️⃣ Example Using ChatPromptTemplate

```
from langchain.prompts import ChatPromptTemplate
chat_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an academic tutor."),
    ("human", "{question}")
])
```

Here:

- System role defines behavior.
- Human role contains the user query.
- The response is conversational.

5️⃣ Where It Is Used

LangChain prompts with users are used in:

- Chatbots
- AI Agents
- Question-answering systems
- RAG (Retrieval-Augmented Generation) systems
- Interactive tutoring systems

6️⃣ Characteristics

- Dynamic prompt structure
- Personalized responses
- Uses placeholders (variables)
- Supports real-time interaction
- Suitable for conversational AI

7️⃣ Advantages

- Flexible and reusable
- Handles different user queries
- Enables personalization
- Essential for real-world AI applications

8️⃣ Limitations

- Slightly more complex than static prompts
- Requires proper variable handling
- Needs validation of user input

✅ Final Exam Definition

LangChain Prompts with Users are dynamic prompt templates that incorporate user queries at runtime by replacing predefined placeholders, enabling interactive and personalized responses from Large Language Models.

📘 LangChain Chains

1️⃣ Definition

A LangChain Chain is a sequence of connected components that work together to complete a task using a Large Language Model (LLM). The output of one component becomes the input of the next component.

In simple terms: A chain links multiple steps into a structured AI workflow.

2️⃣ Purpose of Chains

Chains are used to:

- Organize multiple operations in order
- Connect prompts with LLMs
- Combine retrieval with generation (RAG)
- Add memory to conversations
- Build complete AI pipelines

Instead of calling an LLM directly, chains allow structured processing.

3️⃣ How a Chain Works

Basic flow:

User Input → Prompt Template → LLM → Output

In advanced systems:

User Input → Retriever → Context + Prompt → LLM → Final Output

Each step is connected sequentially.

4️⃣ Types of LangChain Chains

1. LLMChain

Used to connect a PromptTemplate with an LLM.

2. SequentialChain

Executes multiple chains in sequence. Output of first chain becomes input to second chain.

3. SimpleSequentialChain

Simplified version of SequentialChain for straightforward workflows.

4. RetrievalQA Chain

Used in RAG systems. Combines a retriever and LLM to answer questions using external data.

5️⃣ LLMChain (Basic Chain)

LLMChain connects a PromptTemplate with an LLM.

Example:

```
from langchain.prompts import PromptTemplate
from langchain.llms import OpenAI
from langchain.chains import LLMChain

# Step 1: Create Prompt
prompt = PromptTemplate(
input_variables=["topic"],
template="Explain the concept of {topic} in simple terms."
)
# Step 2: Initialize LLM
llm = OpenAI(temperature=0.7)

# Step 3: Create Chain
chain = LLMChain(llm=llm, prompt=prompt)

# Step 4: Run Chain
response = chain.run("Machine Learning")
print(response)
```

6️⃣ Where Chains Are Used

LangChain Chains are used in:

- Chatbots
- AI Agents

- RAG systems
- Document question answering
- Multi-step reasoning systems

7️⃣ Advantages

- Modular design
- Reusable components
- Structured workflow
- Easy debugging
- Scalable architecture

8️⃣ Limitations

- More complex than single LLM calls
- Requires proper configuration
- Debugging multi-step chains can be challenging

✅ Final Exam Definition

LangChain Chains are structured pipelines that connect multiple components such as prompts, Large Language Models, retrievers, and memory modules to execute multi-step AI tasks in an organized and sequential manner.